LAPORAN JOBSHEET
“17 - Implementasi Login Google Provider dengan NextAuth.js + Firebase”

MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

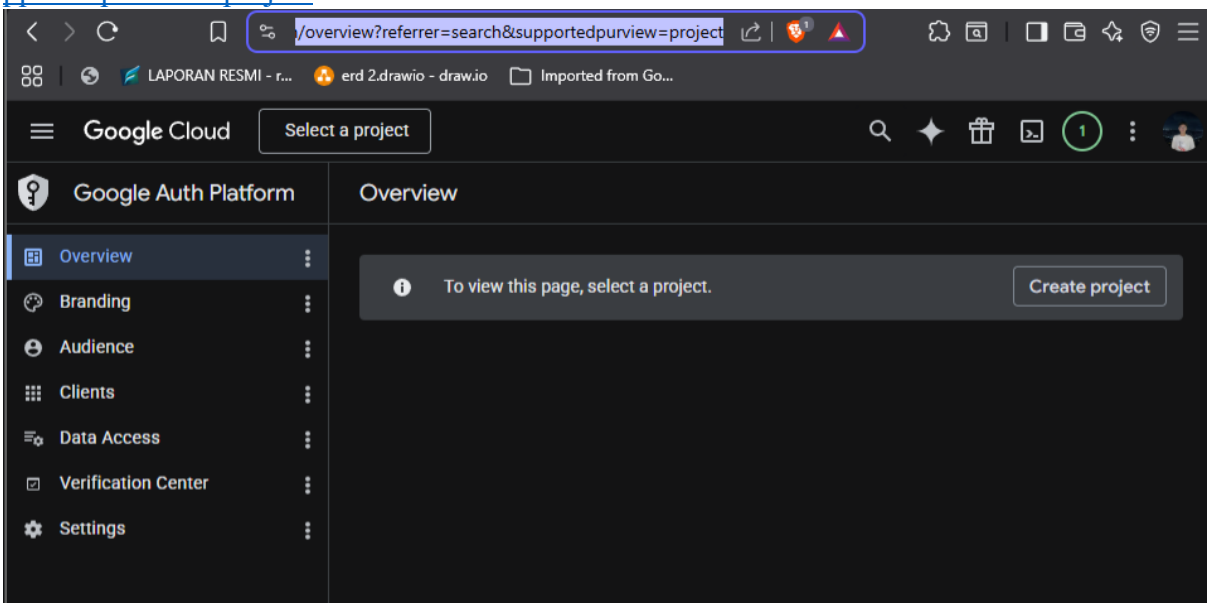
POLITEKNIK NEGERI MALANG

2026

PRAKKTIKUM

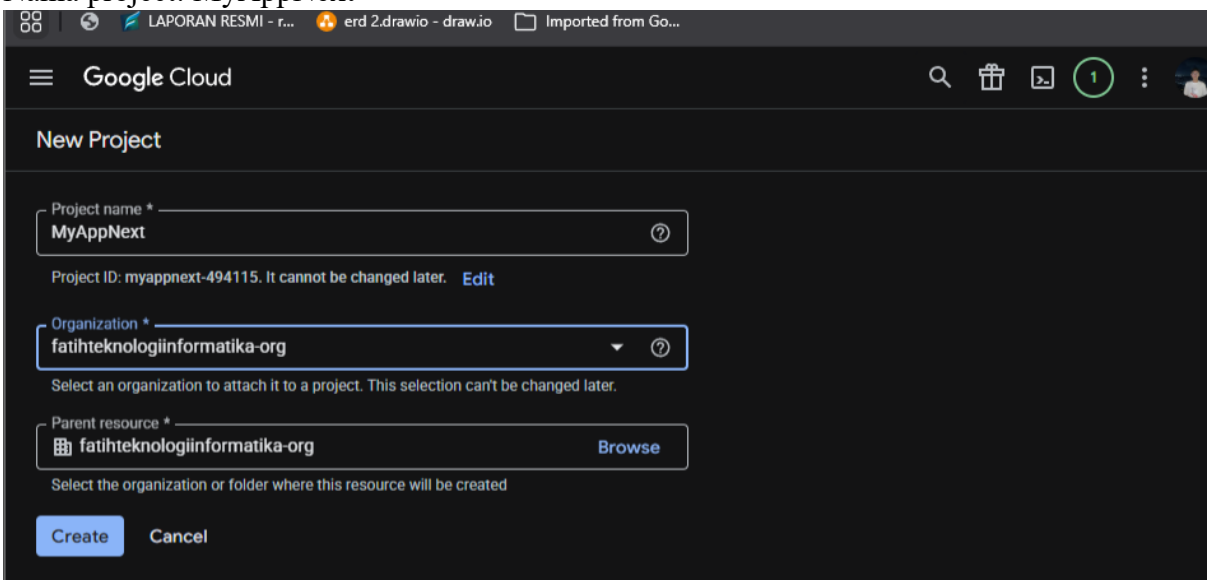
Langkah 1 – Masuk ke Google Cloud Console Buka:

https://console.cloud.google.com/projectselector2/auth/overview?referrer=search&supportedpurview=project



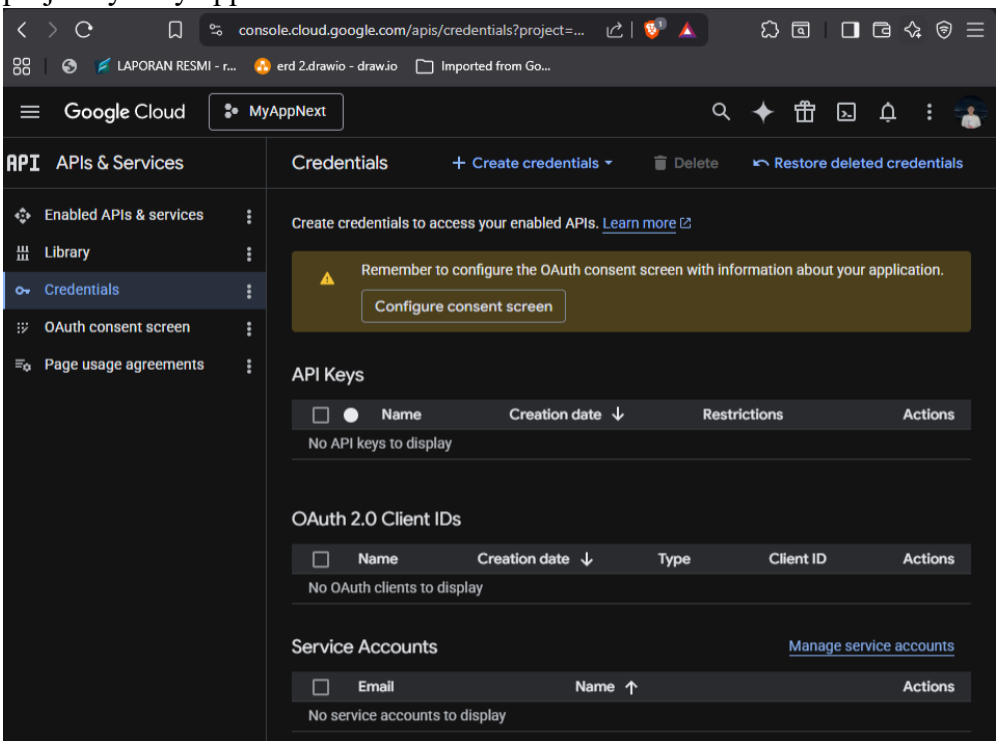
Langkah 2 – Buat Project Baru

1. Klik New Project
2. Nama project: MyAppNext



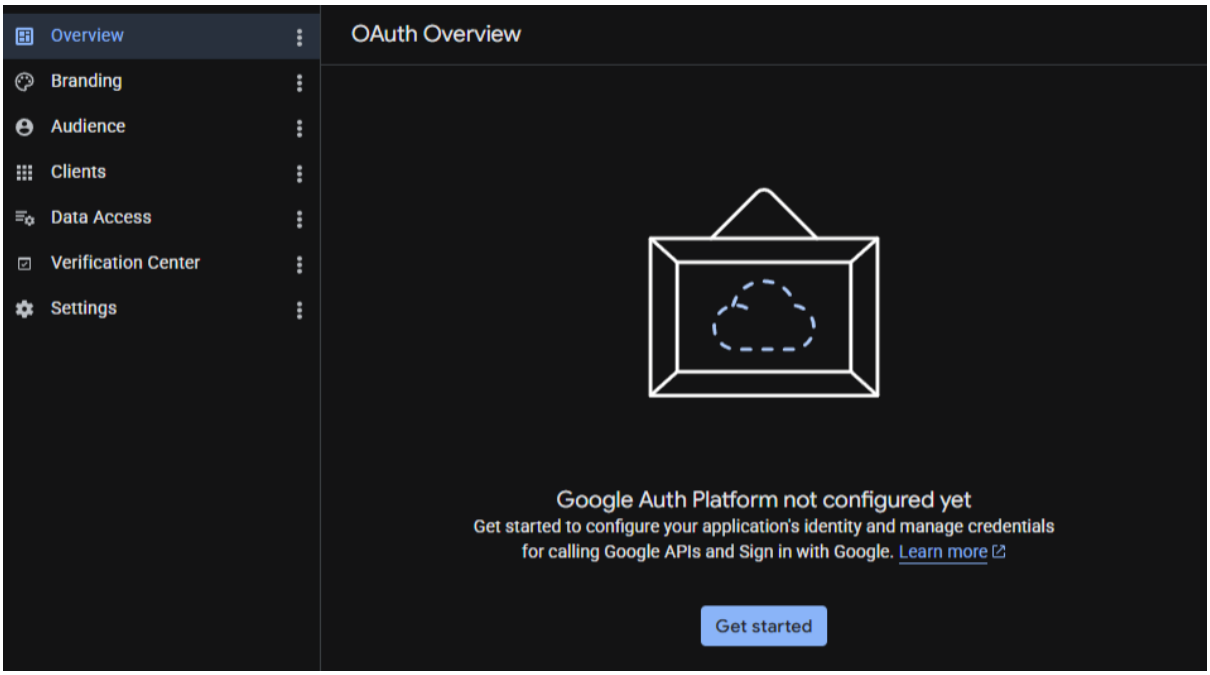
3. Klik Create

4. Setelah berhasil klik https://console.cloud.google.com/apis/credentials pastikan projectnya MyAppNext

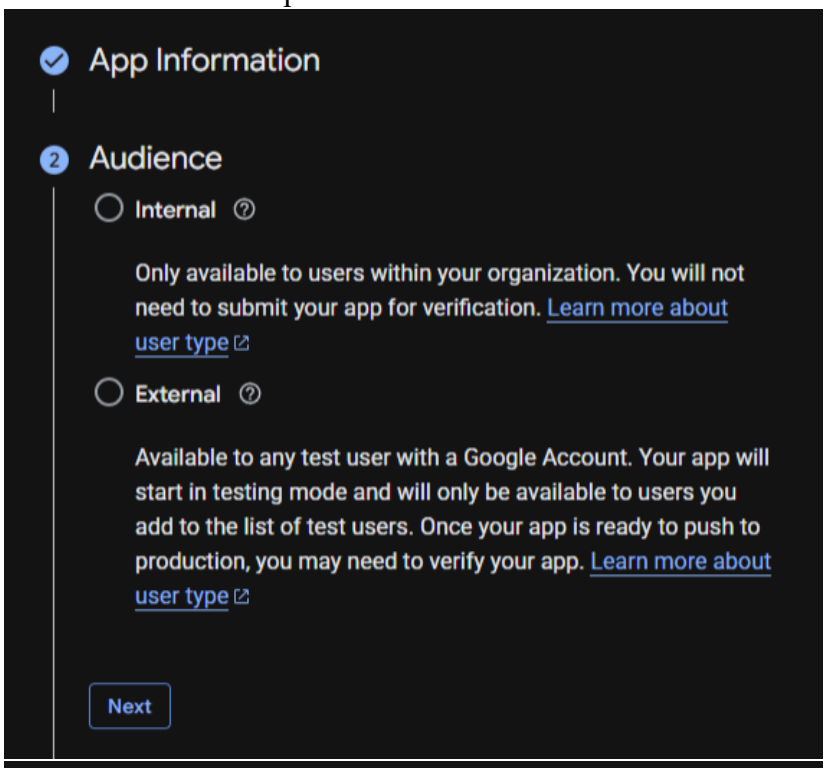


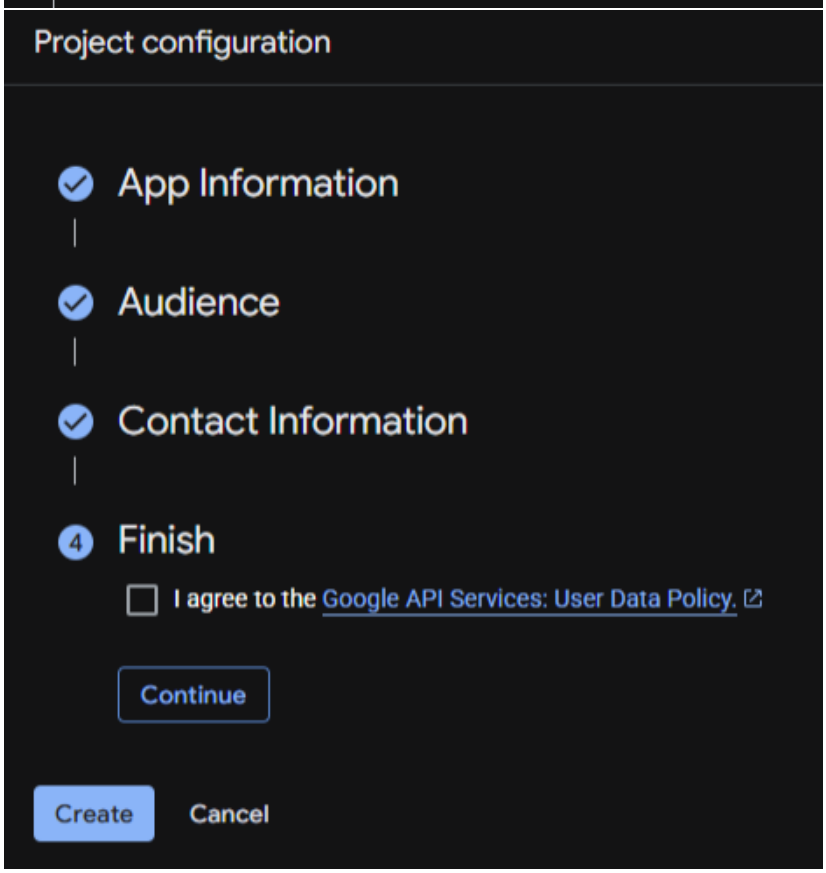
Langkah 3 – Konfigurasi OAuth Consent Screen

1. Pilih OAuth consent screen
2. Pilih Get Started



3. Maka akan muncul seperti berikut dan isikan

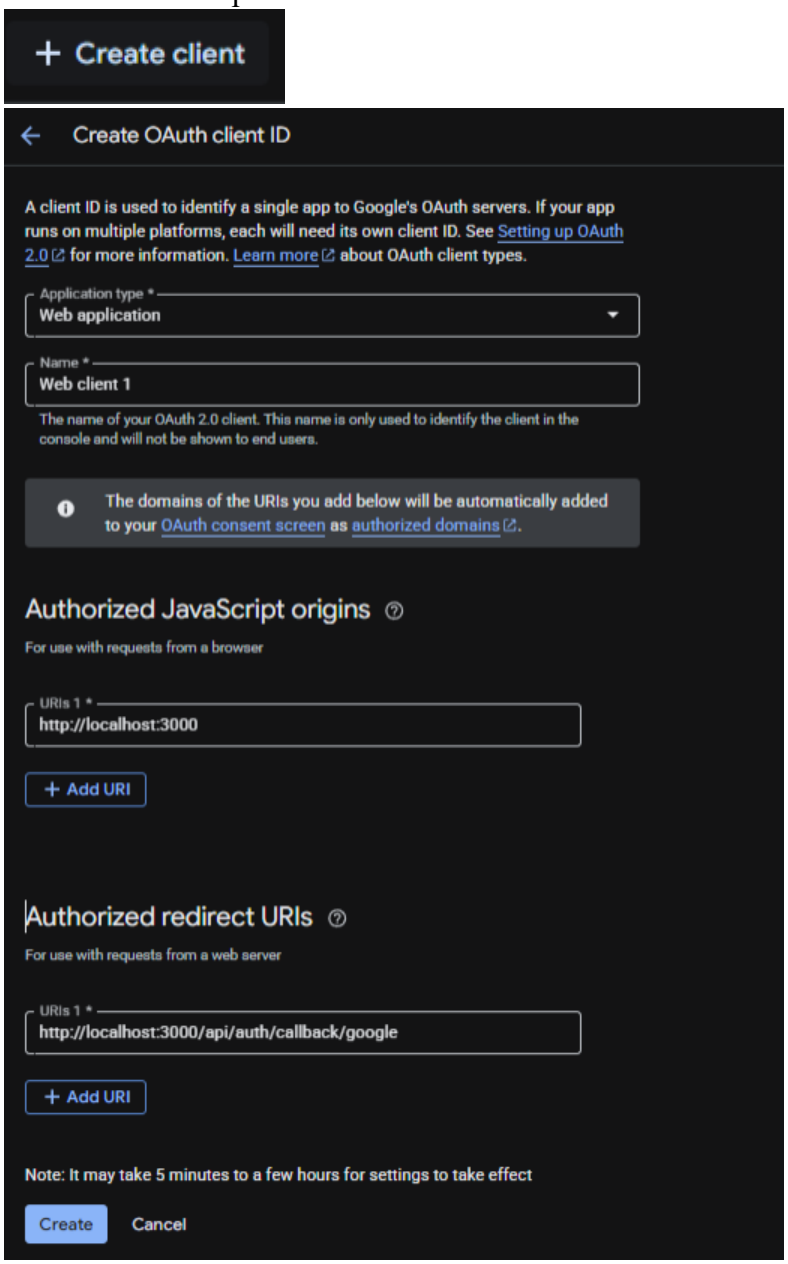




Klik create

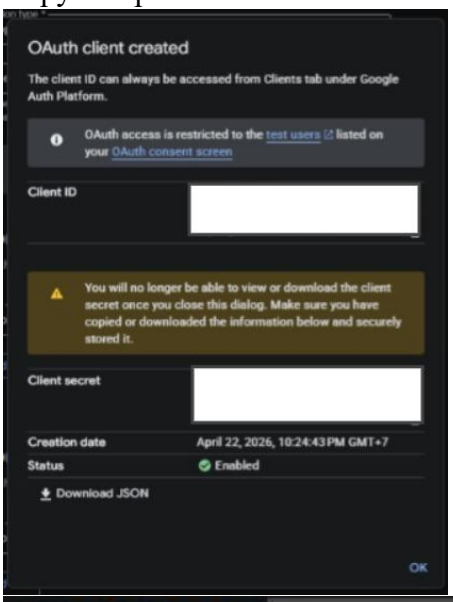
Langkah 4 – Buat OAuth Credentials

1. Klik create client pada Clients



TAMBAHKAN ENVIRONMENT VARIABLES

1. Copy dan paste client ID dan Client secret ke .env



KONFIGURASI GOOGLE PROVIDER DI NEXTAUTH DAN HANDLE CALLBACK JWT & SESSION

1. Buka file [...nextauth].ts pada folder api/auth dan modifikasi menjadi berikut

```
export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt",
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({ ···
    }),

    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID || "",
      clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
    }),
  ],
```

```
  callbacks: {
    async jwt({ token, account, profile, user }) {
      if (account?.provider === "credentials" && user) {
        token.id = user.id;
        token.email = user.email;
        token.fullName = user.fullName;
        token.role = user.role;
      }

      // Jika login dengan Google, tambahkan informasi yang diperoleh ke token
      if (account?.provider === "google") {
        const data = {
          fullname: user?.name,
          email: user?.email,
          image: user?.image,
          type: account.provider,
        };
        token.fullname = data.fullname;
        token.email = data.email;
        token.image = data.image;
        token.type = data.type;
      }
      return token;
    },
```

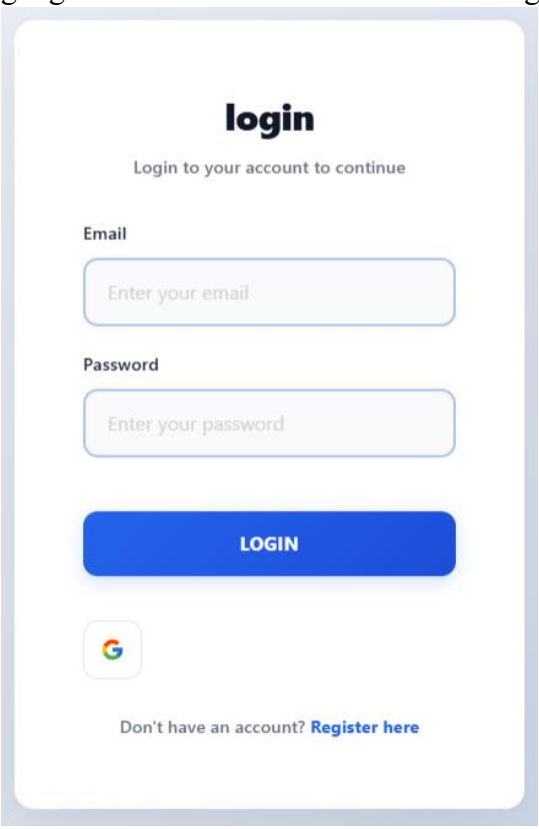
```
async session({ session, token }: any) {
  if (token.id) {
    session.user.id = token.id;
  }
  if (token.email) {
    session.user.email = token.email;
  }
  if (token.fullName) {
    session.user.fullName = token.fullName;
  }
  if (token.role) {
    session.user.role = token.role;
  }
  if (token.image) {
    session.user.image = token.image;
  }
  if (token.type) {
    session.user.type = token.type;
  }
```

TAMBAHKAN BUTTON GOOGLE

1. Modifikasi file index.tsx pada folder views/auth/login

```
<button
  type="button"
  className={styles.google_login_button}
  onClick={() => signIn("google", { callbackUrl, redirect: false })}
  disabled={isLoading}
  aria-label="Sign in with Google"
  title="Sign in with Google"
>
  <FcGoogle size={20} />
</button>
```

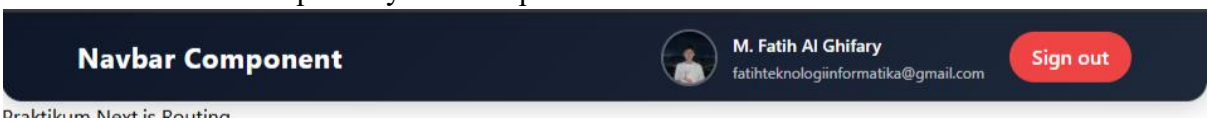
2. Jalankan browser http://localhost:3000/auth/login masuk melalui sign in with google.Jika berhasil maka akan terhubung dengan akun google.



3. Menampilkan image dari google
   - Buka file components/navbar/index.tsx dan tambahkan code berikut

```
{session.user.image && (
  <Image
    src={session.user.image}
    alt={session.user.fullName || "User Avatar"}
    width={40}
    height={40}
    className={styles.navbar__avatar}
  />
)}
```

   - Jika berhasil maka tampillannya akan seperti berikut



SIMPAN DATA GOOGLE KE DATABASE

1. Buka file servicefirebase.ts pada folder src/utils/db/ dan tambahkan beberapa kode berikut

```
addDoc,
updateDoc,
```

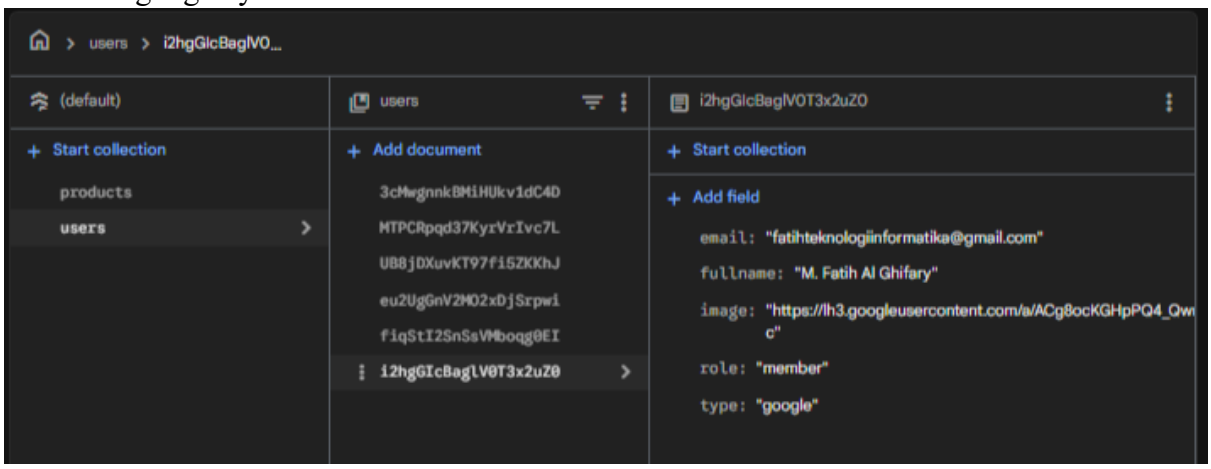
2. Menambahkan kode lagi sebagai berikut

```
export async function signInWithGoogle(userData: any, callback: any) {
  try {
    // Check if user with this email already exists
    const q = query(
      collection(db, "users"),
      where("email", "==", userData.email),
    );
    const querySnapshot = await getDocs(q);
    const data:any = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    if (data.length > 0) {
      // User already exists, return existing user data
      userData.role = data[0].role || "member"; // Pastikan role tetap terisi
      callback({
        status: "success",
        message: "User created and signed in with Google successfully",
        data: userData,
      });
    } else {
      // User doesn't exist, create new user
      userData.role = "member"; // Set default role untuk pengguna baru
      await addDoc(collection(db, "users"), userData);
      callback({
        status: "success",
        message: "User created and signed in with Google successfully",
        data: userData,
      });
    }
  } catch (error: any) {
    console.error("Error during Google sign in:", error);
    callback({
      status: "error",
      message: "An error occurred during Google sign in. Please try again.",
      error: error,
    });
  }
}
```

3. Panggil Service di JWT Callback buka file […nextAuth].ts

```
callbacks: {
  async jwt({ token, account, profile, user }) {
    if (account?.provider === "credentials" && user) {
      token.id = user.id;
      token.email = user.email;
      token.fullName = user.fullName;
      token.role = user.role;
    }

    // Jika login dengan Google, tambahkan informasi yang diperoleh ke token
    if (account?.provider === "google") {
      const data = {
        fullname: user?.name,
        email: user?.email,
        image: user?.image,
        type: account.provider,
      };
      await signInWithGoogle(data, (response: any) => {
        if (response.status === "success") {
          token.id = response.data.id;
          token.email = response.data.email;
          token.fullName = response.data.fullName;
          token.role = response.data.role;
          token.image = response.data.image;
          token.type = response.data.type;
        } else {
          console.error("Google sign-in failed:", response.message);
        }
      });
    }
```

4. Jalankan browser dan login menggunakan akun google setelah cek di firebase, jika data akun googlenya masuk ke database maka anda telah berhasil



ANALISIS DAN DISKUSI

1. **Apa perbedaan login credential dan login Google?**
   Login credential menggunakan email dan password yang dibuat serta dikelola sendiri oleh sistem (disimpan di database dan perlu proses validasi serta hashing), sedangkan login Google menggunakan akun pihak ketiga melalui OAuth sehingga proses autentikasi dilakukan oleh Google dan aplikasi hanya menerima data user yang sudah terverifikasi tanpa perlu menyimpan password.

2. **Mengapa data Google tetap perlu disimpan ke database?**
   Walaupun autentikasi dilakukan oleh Google, data user tetap perlu disimpan di database (Firebase) agar aplikasi dapat mengenali user tersebut di sistem, menyimpan informasi tambahan (seperti role), serta mengelola data yang berkaitan dengan user di dalam aplikasi.

3. **Apa fungsi JWT callback?**
   JWT callback pada NextAuth berfungsi untuk memproses dan menyimpan data user ke dalam token saat login, termasuk memanggil service untuk menyimpan data user ke database (Firebase), sehingga data tersebut bisa digunakan kembali pada session tanpa harus login ulang.

4. **Mengapa perlu multi-role?**
   Multi-role diperlukan agar sistem dapat membedakan hak akses setiap user (misalnya admin dan user biasa), sehingga fitur yang ditampilkan atau diakses bisa disesuaikan dengan peran masing-masing pengguna.

5. **Apa risiko jika tidak menyimpan user ke database?**
   Jika user tidak disimpan ke database, maka aplikasi tidak bisa mengenali user secara konsisten, tidak dapat menambahkan informasi penting seperti role atau data tambahan, serta berisiko menyebabkan masalah dalam pengelolaan data dan fitur (misalnya user dianggap baru setiap login).

TUGAS MANDIRI

1. Tambahkan role editor

```ts
const hanyaAdmin = ["/admin"];
const hanyaEditor = ["/editor"];

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = []
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const loginUrl = new URL("/", req.url);
        loginUrl.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(loginUrl);
      }
      if (token.role !== "admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
      }
      if (
        token.role !== "editor" &&
        token.role !== "admin" &&
        hanyaEditor.includes(pathname)
      ) {
        return NextResponse.redirect(new URL("/", req.url));
      }
    }
```

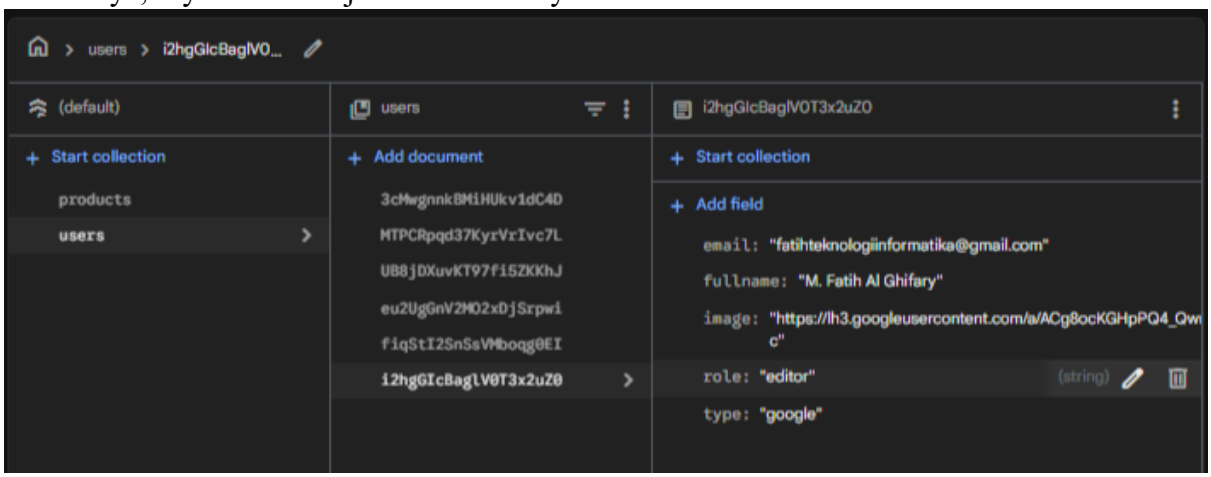
Didalam file middleware.ts:

```ts
const editorRoutes = ["/editor"];
```

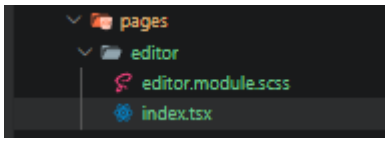
```ts
// Cek apakah route saat ini adalah editor
const isEditorRoute = editorRoutes.some((route) => pathname.startsWith(route));
```

```ts
// Jika mengakses editor route, cek apakah user memiliki role editor atau admin
if (isEditorRoute) {
  if (!token) {
    // Jika belum login, redirect ke /auth/login
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
  if (token.role !== "editor" && token.role !== "admin") {
    // Jika sudah login tapi bukan editor atau admin, redirect ke home
    return NextResponse.redirect(new URL("/", request.url));
  }
}
```

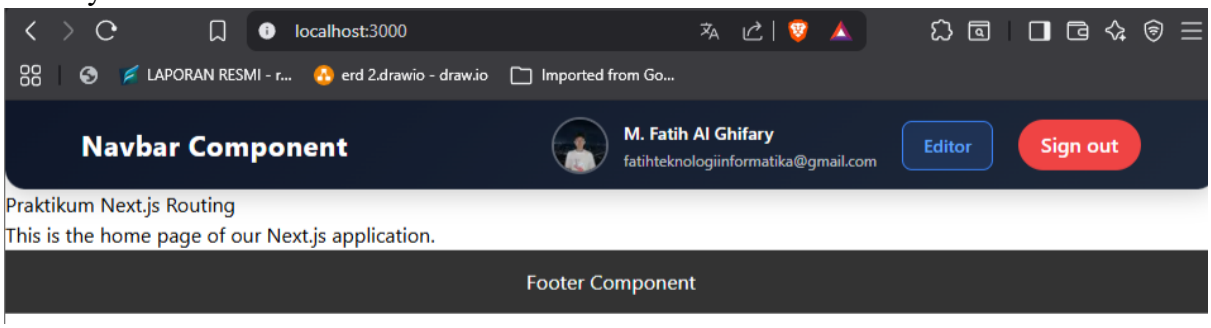
Akun saya, saya ubah menjadi editor rolenya:



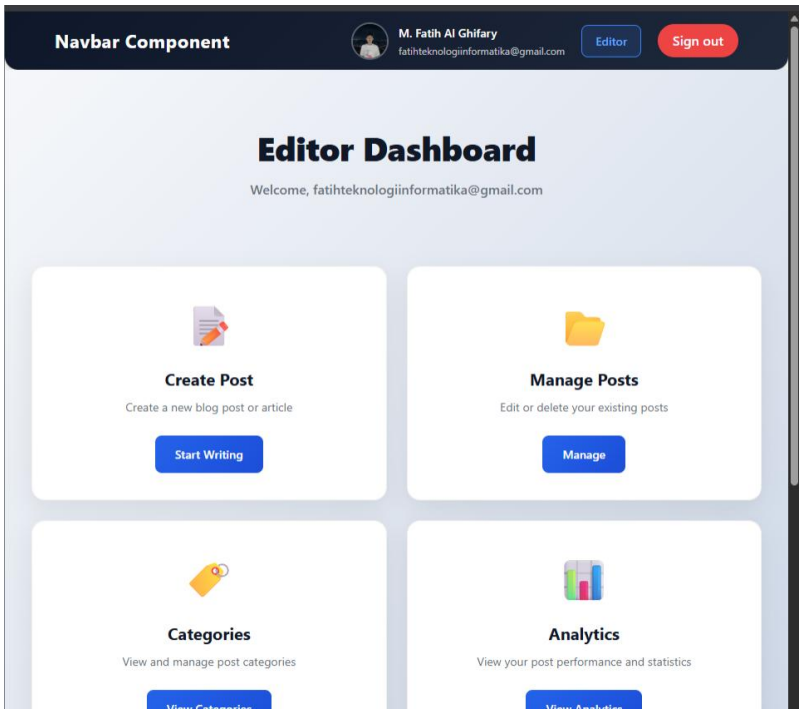
2. Buat halaman khusus editor



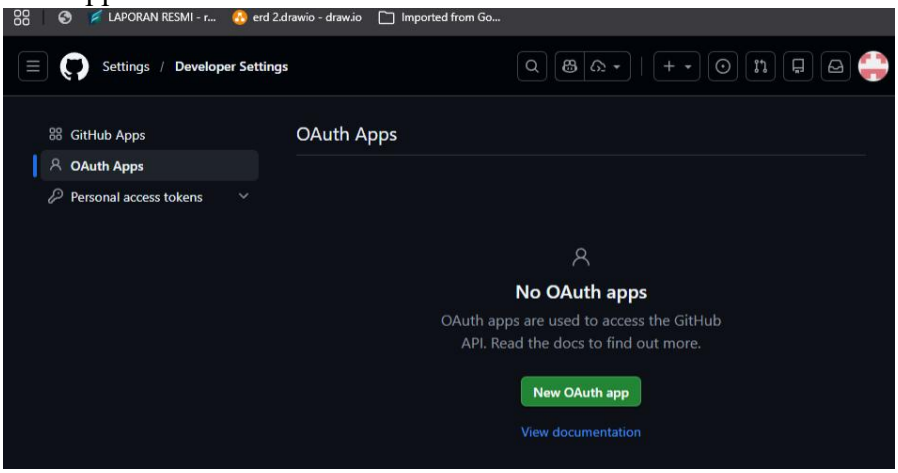
Hasilnya:



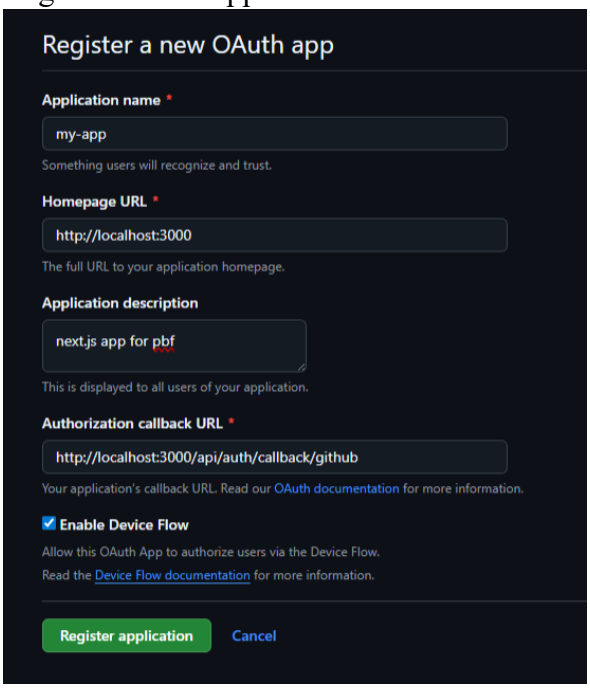
Maka akan muncul opsi untuk masuk ke halaman editor, jika di klik maka akan direct ke halaman editor



3. Tambahkan provider GitHub
   - Buat app



- Register OAuth app



- Copy Client ID dan Client Secret, kemudian paste di .env.local/.env
- Kemudian tambahkan Github provider di nextauth.ts

```
    GithubProvider({
      clientId: process.env.GITHUB_CLIENT_ID || "",
      clientSecret: process.env.GITHUB_CLIENT_SECRET || "",
    }),
  ],
```

- Menambahkan function signInWithGithub di servicesfirebase.ts

```
export async function signINwithGithub(userData: any, callback: any) {
  try {
    // Check if user with this email already exists
    const q = query(
      collection(db, "users"),
      where("email", "==", userData.email),
    );
    const querySnapshot = await getDocs(q);
    const data:any = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    if (data.length > 0) {
      // User already exists, return existing user data
      userData.role = data[0].role || "member"; // Pastikan role tetap terisi
      callback({
        status: "success",
        message: "User created and signed in with Github successfully",
        data: userData,
      });
    } else {
      // User doesn't exist, create new user
      userData.role = "member"; // Set default role untuk pengguna baru
      await addDoc(collection(db, "users"), userData);
      callback({
        status: "success",
        message: "User created and signed in with Github successfully",
        data: userData,
      });
    }
  } catch (error: any) {
    console.error("Error during Github sign in:", error);
    callback({
      status: "error",
      message: "An error occurred during Github sign in. Please try again.",
      error: error,
    });
  }
}
```

- Tambahkan callback di nextauth.ts

```
if (account?.provider === "github") {
  const data = {
    fullname: user?.name,
    email: user?.email,
    image: user?.image,
    type: account.provider,
  };
  await signInWithGithub(data, (response: any) => {
    if (response.status === "success") {
      token.id = response.data.id;
      token.email = response.data.email;
      token.fullName = response.data.fullName;
      token.role = response.data.role;
      token.image = response.data.image;
      token.type = response.data.type;
    } else {
      console.error("Github sign-in failed:", response.message);
    }
  });
}
return token;
},
```

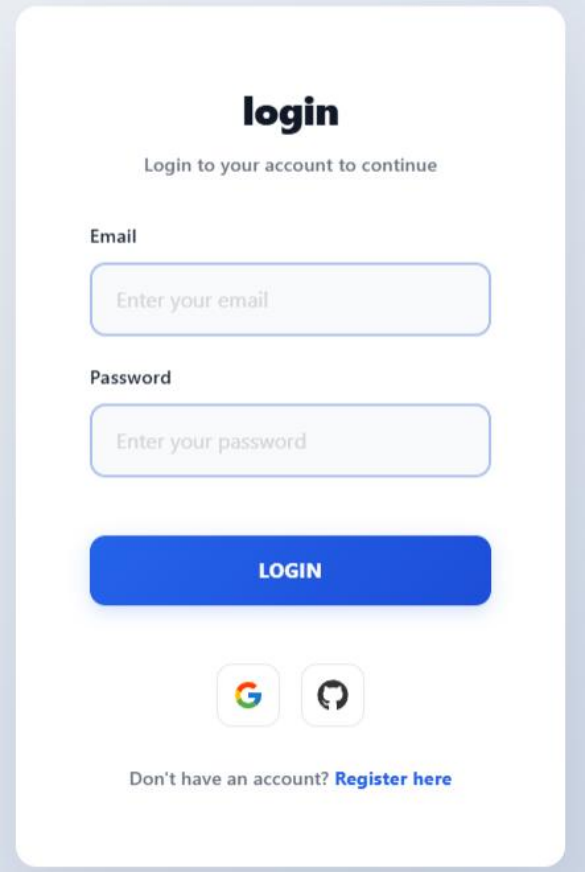
- Menambahkan button github di views/auth/login/index.tsx

```
<button
  type="button"
  className={styles.github_login_button}
  onClick={() => signIn("github", { callbackUrl, redirect: false })}
  disabled={isLoading}
  aria-label="Sign in with Github"
  title="Sign in with Github"
>
  <FaGithub size={20} />
</button>
```

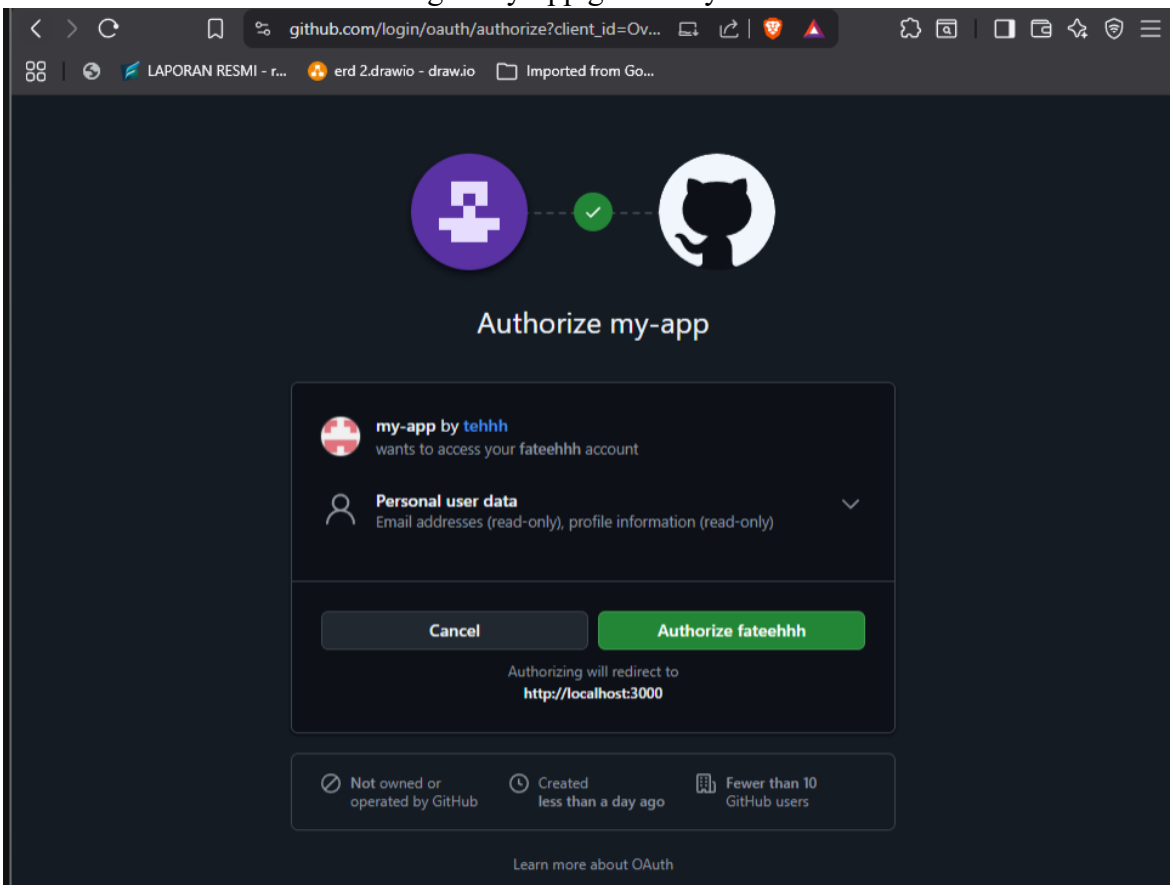
- Tambah sedikit design di login.module.scss

```
.github_login_button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 3rem;
  height: 3rem;
  margin-top: 1rem;
  border: 1.5px solid #e5e7eb;
  border-radius: 0.75rem;
  background-color: white;
  cursor: pointer;
  transition: all 0.3s ease;
  padding: 0;

  &:hover {
    border-color: #d1d5db;
    background-color: #f9fafb;
    transform: translateY(-2px);
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
  }

  &:active {
    transform: translateY(0);
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.04);
  }

  &:disabled {
    cursor: not-allowed;
    opacity: 0.5;
  }
}
```

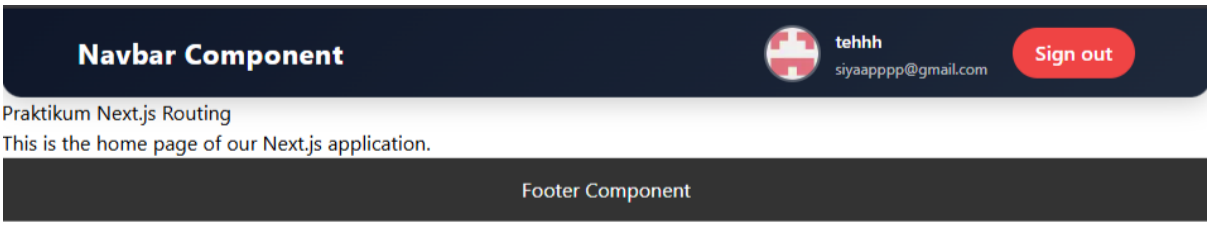
- Hasilnya ketika akses http://localhost:3000/auth/login



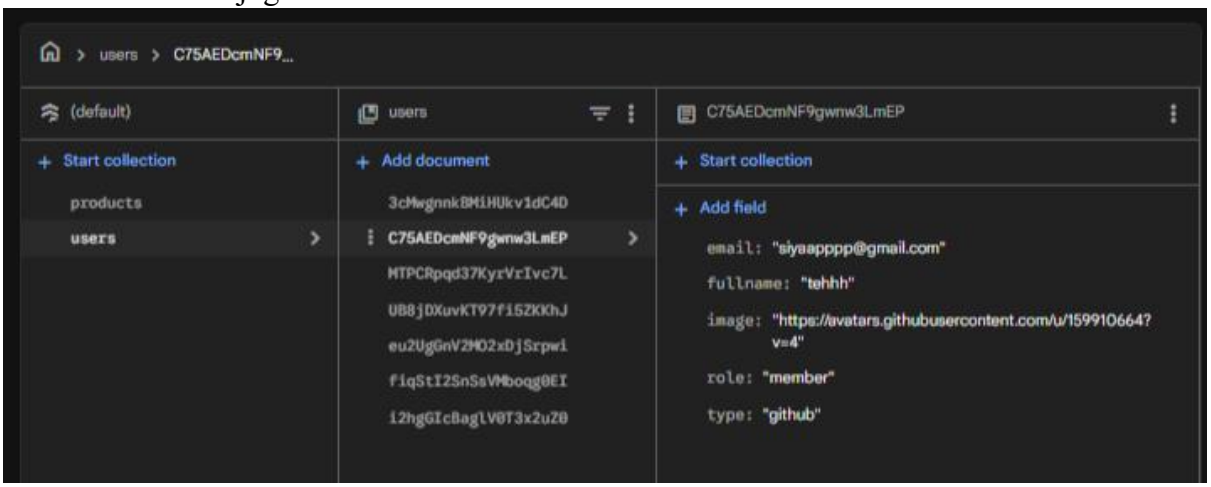
- Hasil sudah bisa terauthorize dengan my-app github saya



- Ketika sudah terauthorized maka akan masuk ke direct ke home



- Dan sudah masuk juga di database firestone



4. Gunakan next/image untuk optimasi avatar

```
import Image from "next/image";
import Link from "next/link";
```

```
<Image
  src={
    session.user.image ||
    `https://ui-avatars.com/api/?name=${encodeURIComponent(
      session.user.name ||
        session.user.fullName ||
        session.user.email ||
        "User",
    )}&background=random&color=fff`
  }
  alt={session.user.fullName || "User Avatar"}
  width={45}
  height={45}
  className={styles.navbar__avatar}
/>
```